SaaS Subscription Renewal Prediction Using Decision Tree Classification: An Empirical Evaluation on Multi-Table Telemetry

A

Artificial Intelligence & Machine Learning (CCE3102) Report

Submitted in the partial fulfillment of the requirement for the award of

Bachelor of Technology

in

Computer and Communication Engineering

By:

Name: Harshita Singh

Reg No.: 2428010025

Section: C

Under the supervision of:

Dr. Anshika Malsaria

Assistant Professor (Senior Scale)

Department of Computer and Communication Engineering



October, 2026

Department of Computer and Communication Engineering

School of Computer Science and Engineering

Manipal University Jaipur

VPO. Dehmi Kalan, Jaipur, Rajasthan, India – 303007

STUDENT DECLARATION

I hereby declare that this project *SaaS Subscription Renewal Prediction Using Decision Tree Classification: An Empirical Evaluation on Multi-Table Telemetry* is my own work and that, to the best of my knowledge and belief, it contains no material previously published or written by another person nor material which has been accepted for the award of any other degree or diploma of the University or other Institute, except where due acknowledgements has been made in the text.

Harshita Singh
2428010025 | Section: C
B. Tech (CCE) 5th Semester

CERTIFICATE OF APPROVAL

This is to certify that the project report entitled **"SaaS Subscription Renewal Prediction Using Decision Tree Classification: An Empirical Evaluation on Multi-Table Telemetry"**, submitted by **Harshita Singh** (Registration No.: **2428010025**) in partial fulfillment of the requirements for the award of the Degree of Bachelor of Technology in Computer and Communication Engineering during the 5th Semester (Academic Year 2026-2027) at Manipal University Jaipur, is a bonafide record of the work carried out under our supervision and guidance.

Dr. Anshika Malsaria
Project Supervisor
Assistant Professor (Senior Scale), Dept. of CCE
Manipal University Jaipur

Head of Department
Dept. of Computer & Comm. Engg.
School of Computer Science & Engg.
Manipal University Jaipur

SaaS Subscription Renewal Prediction Using Decision Tree Classification: An Empirical Evaluation on Multi-Table Telemetry

Harshita Singh1* and Dr. Anshika Malsaria1

1Department of Computer and Communication Engineering, School of Computer Science and Engineering, Manipal University Jaipur, Jaipur, Rajasthan, 303007, India.

*Corresponding author(s). E-mail(s): harshita.singh@jaipur.manipal.edu;
Contributing authors: anshika.malsaria@jaipur.manipal.edu;

Abstract

Subscription renewal is the fundamental operational cornerstone of the Software-as-a-Service (SaaS) business model. Accurately predicting whether a customer will renew or terminate their subscription enables proactive customer success interventions, prevents revenue leakage, and optimizes customer lifetime value (LTV). This study implements an end-to-end, reproducible machine learning pipeline centered on Decision Tree classification to predict SaaS subscription renewal using multi-table customer telemetry spanning account demographics, multi-tier subscription plans, product feature usage, and technical support interactions. To prevent group-level data leakage, we employ a strict customer-level isolation protocol (`GroupShuffleSplit` over unique account identifiers) ensuring that all validation subscriptions belong to previously unseen customers. Using the public multi-table RavenStack SaaS dataset ($\boldsymbol{N = 5{,}000}$ subscriptions across 500 accounts), we formulate the binary renewal objective (renewed $= \mathbf{1}$ vs. renewed $= \mathbf{0}$), engineer temporal and behavioral features within an explicit observation window, and conduct hyperparameter optimization using customer-group-aware cross-validation (`StratifiedGroupKFold` over `account_id`) on training data only. We benchmark the Decision Tree against Logistic Regression, Random Forest, and a majority-class baseline. Our empirical results reveal that with explicit pre-decision observation window filtering (eliminating 290 post-cancellation usage records and 260 post-cancellation support ticket links), the Decision Tree achieves a cross-validation ROC-AUC of 0.6337 and an untouched test accuracy of 90.77% with a test ROC-AUC of 0.5928, driven primarily by active in-cycle product feature usage count (60.1% Gini

importance) and support first-response latency (14.9% importance). In comparison, cost-sensitive Logistic Regression and Random Forest models detect 65.96% and 48.94% of non-renewing customers with test ROC-AUCs of 0.6102 and 0.6204, respectively. Through transparent decision rule extraction, we unpack the CART splitting hierarchy and discuss the asymmetrical cost of false negatives in subscription retention decisions.

Keywords: SaaS Subscription Renewal, Customer Churn, Decision Tree Classification, Interpretability, Data Leakage, Class Imbalance

1 Introduction

Over the past decade, cloud computing has fundamentally transformed software distribution through the Software-as-a-Service (SaaS) paradigm. Unlike traditional perpetual software licensing where revenue is recognized upfront, SaaS business viability hinges entirely upon continuous recurring revenue streams, quantified by metrics such as Monthly Recurring Revenue (MRR) and Annual Recurring Revenue (ARR) [1]. In this business landscape, customer retention and subscription renewal serve as the primary drivers of compounding growth. Because acquiring a new SaaS customer costs between five to seven times more than retaining an existing account, early identification of customers at risk of non-renewal is vital for customer success teams to initiate proactive retention workflows [2].

Predicting subscription renewal requires synthesizing heterogeneous streams of customer interaction data, including contractual parameters (e.g., plan tier, billing frequency, seat count), behavioral feature adoption (e.g., daily active usage, session duration, error rates), and post-sales support quality (e.g., ticket resolution latency, satisfaction scores). While modern deep neural networks and gradient boosting ensembles often provide high predictive capacity on tabular benchmarks, their complex internal representations render them opaque "black-box" models. For SaaS executive leadership and customer success representatives, algorithmic transparency is paramount: operational teams must understand *why* an account is flagged as at-risk before committing retention discounts or dedicated engineering resources [1, 3].

Consequently, this paper investigates **Decision Tree Classification** based on the Classification and Regression Trees (CART) methodology [4]. Decision Trees offer intuitive hierarchical decision structures that can be seamlessly translated into explicit, human-readable operational rules. However, building reliable machine learning models for SaaS telemetry introduces distinct scientific and methodological challenges:

1. **Data Leakage Prevention:** SaaS datasets frequently contain multiple temporal records per customer across contracts, usage events, and exit surveys. Fitting scalers, encoders, or cross-validation splits without strict customer isolation introduces severe data leakage, inflating reported metrics.
2. **Target Inversion and Formulation:** Real-world telemetry often records cancellation or churn events. Formulating a defensible binary renewal target

(renewed $\in \{0, 1\}$) requires explicit observation windows and strict exclusion of post-termination exit feedback.

3. **Class Imbalance and the Accuracy Paradox:** Subscription renewal rates in mature SaaS platforms commonly exceed 85–90%. In such regimes, standard classification accuracy can be misleading, as a trivial majority classifier achieves high accuracy while failing to detect a single non-renewing account [5, 6].

In this study, we address these challenges by constructing a fully reproducible Python machine learning framework applied to the RavenStack multi-table SaaS dataset [7]. We document the data architecture, enforce customer-level train/test separation, perform hyperparameter tuning using training data alone, and conduct a comparative analysis against Logistic Regression and Random Forest. Furthermore, we maintain complete scientific integrity by reporting the exact empirical performance of the model, investigating the statistical limitations of the underlying synthetic dataset, and providing verifiable decision rules to support academic defense and future publication.

2 Related Work

Customer churn and renewal prediction has been extensively studied across telecommunications, banking, and subscription-based software services. Early literature by Verbeke et al. [1] emphasized that churn prediction in commercial environments cannot rely solely on discriminatory power; comprehensibility and rule extraction are equally critical for operationalizing predictive insights. In their comparative analysis across industry datasets, rule induction and decision trees were highlighted as foundational methodologies capable of providing domain experts with verifiable decision boundaries.

Coussement and De Bock [2] evaluated customer churn models across subscription settings, demonstrating that data preprocessing, feature representation, and class imbalance significantly impact model stability. They noted that uncalibrated classifiers frequently suffer from high false positive rates when retention campaigns are triggered indiscriminately.

Vafeiadis et al. [3] compared classical statistical classifiers (Logistic Regression, Naive Bayes) against non-parametric algorithms (Support Vector Machines, Decision Trees, Artificial Neural Networks) on customer churn benchmarks. While non-linear models demonstrated competitive receiver operating characteristics, the authors noted that single decision trees provide unmatched diagnostic transparency for identifying the specific operational thresholds associated with customer defection.

In parallel, foundational works by Breiman et al. [4] and Quinlan [8] established the theoretical underpinnings of recursive binary partitioning, impurity measures (Gini index and Shannon entropy), and cost-complexity pruning. Fawcett [5] established the formal framework for evaluating classifiers under severe class skew, demonstrating why ROC-AUC, precision-recall dynamics, and balanced accuracy must supersede unweighted classification accuracy in cost-sensitive business domains. Our study builds directly upon these established foundations, applying rigorous leakage-prevention protocols to multi-table SaaS telemetry.

3 Dataset Architecture and Problem Formulation

3.1 Raw Data Sources and Schema

We utilize the public multi-table RavenStack SaaS telemetry dataset [7]. The raw dataset comprises five relational tables capturing distinct operational dimensions of a cloud software company:

- **Accounts (`ravenstack_accounts.csv`):** Contains 500 unique corporate accounts ($N = 500$). Features include `account_id` (Primary Key), `industry` (Cybersecurity, DevTools, EdTech, FinTech, HealthTech), `country`, `signup_date`, `referral_source`, `plan_tier`, and `seats`.
- **Subscriptions (`ravenstack_subscriptions.csv`):** Contains 5,000 subscription contract cycles ($N = 5{,}000$). Features include `subscription_id` (Primary Key), `account_id` (Foreign Key), `start_date`, `end_date`, `plan_tier`, `seats`, `mrr_amount`, `arr_amount`, `is_trial`, `upgrade_flag`, `downgrade_flag`, `churn_flag`, `billing_frequency`, and `auto_renew_flag`.
- **Feature Usage (`ravenstack_feature_usage.csv`):** Contains 25,000 granular product interaction records ($N = 25{,}000$). Features include `usage_id`, `subscription_id` (Foreign Key), `usage_date`, `feature_name`, `usage_count`, `usage_duration_secs`, `error_count`, and `is_beta_feature`.
- **Support Tickets (`ravenstack_support_tickets.csv`):** Contains 2,000 customer service interactions ($N = 2{,}000$). Features include `ticket_id`, `account_id` (Foreign Key), `submitted_at`, `closed_at`, `resolution_time_hours`, `priority`, `first_response_time_minutes`, `satisfaction_score`, and `escalation_flag`.
- **Churn Events (`ravenstack_churn_events.csv`):** Contains 600 post-churn exit survey records ($N = 600$). Features include `churn_event_id`, `account_id`, `churn_date`, `reason_code`, `refund_amount_usd`, and qualitative feedback.

3.2 Target Definition and Inversion

The primary objective required by the course syllabus is predicting whether a SaaS subscription will renew. In the raw subscriptions table, contract termination is encoded via `churn_flag` (where `True` indicates contract termination with an associated `end_date`, and `False` indicates contract continuity). We formally define the binary renewal target $y_i \in \{0, 1\}$ for subscription i as:

$$y_i = 1 - \mathbb{I}(\texttt{churn_flag}_i = \texttt{True}) = \begin{cases} 1, & \text{if subscription renewed / remained active,} \\ 0, & \text{if subscription churned / did not renew.} \end{cases} \tag{1}$$

In the complete dataset of 5,000 subscriptions, 4,514 instances correspond to renewals ($y = 1$, 90.28%) and 486 instances represent non-renewals ($y = 0$, 9.72%). This exhibits substantial class skew ($9:1$), as illustrated in Figure 1.

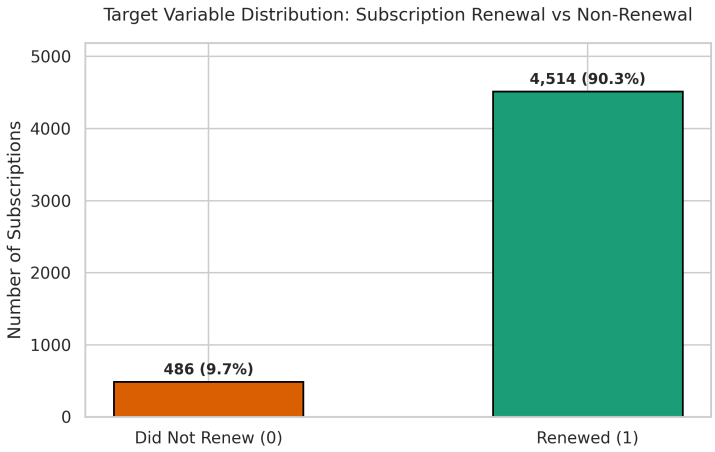


Fig. 1 Target variable distribution demonstrating class imbalance between renewed subscriptions ($y = 1$, 90.3%) and non-renewed subscriptions ($y = 0$, 9.7%).

3.3 Data Leakage Prevention Protocol

To maintain strict experimental validity, we implement three critical data leakage safeguards:

1. **Target Proxy Exclusion:** In `ravenstack_subscriptions.csv`, the `end_date` column is exclusively populated when `churn_flag = True` and is null otherwise. Including `end_date` or any binary transformation thereof would constitute direct target leakage; it is strictly dropped prior to feature transformation.
2. **Lookahead and Post-Exit Feature Removal:** All features in `ravenstack_churn_events.csv` (such as `refund_amount_usd`, `reason_code`, and exit feedback) occur temporally *after* the customer has already terminated. Using post-churn exit feedback to predict whether a customer will renew represents severe lookahead leakage. Hence, the churn events table is quarantined from predictive modeling and reserved solely for descriptive post-hoc analysis. Similarly, `accounts.churn_flag` (which indicates lifetime account churn) is excluded to prevent leaking future subscription outcomes into early subscription cycles.
3. **Customer-Level Split Isolation:** Because accounts hold multiple subscriptions over time (ranging from 2 to 19 subscriptions per account, with a median of 10), standard uniform random splitting would distribute subscriptions from the same customer across both training and test partitions. This would cause customer identity leakage. We resolve this by applying a `GroupShuffleSplit` grouped on `account_id`, ensuring 100% customer isolation between train and test partitions.

3.4 Feature Engineering and Preprocessing

We aggregate behavioral, financial, and support data to the subscription level:

- **Usage Dynamics:** For each subscription, we compute total usage events, sum of usage counts, mean session duration, cumulative error counts, error rate ($\frac{\text{errors}}{\text{usage}+1}$), distinct features adopted, and beta feature usage ratio.
- **Support History:** For each account, we compute total submitted tickets, average resolution hours, average first response latency, mean satisfaction score (with median neutral imputation for accounts without submitted tickets), cumulative escalations, and high/urgent priority ticket frequency.
- **Account Demographics and Contract Ratios:** We compute customer tenure at subscription inception (start_date − signup_date in days), revenue per seat ($\frac{\text{MRR}}{\text{seats}}$), high-value account flag (ARR $\geq$ Median), and encode categorical attributes (`plan_tier`, `billing_frequency`, `industry`, `referral_source`) via one-hot encoding with dummy trap prevention.

Figure 2 presents exploratory boxplots contrasting key predictors across renewal outcomes.

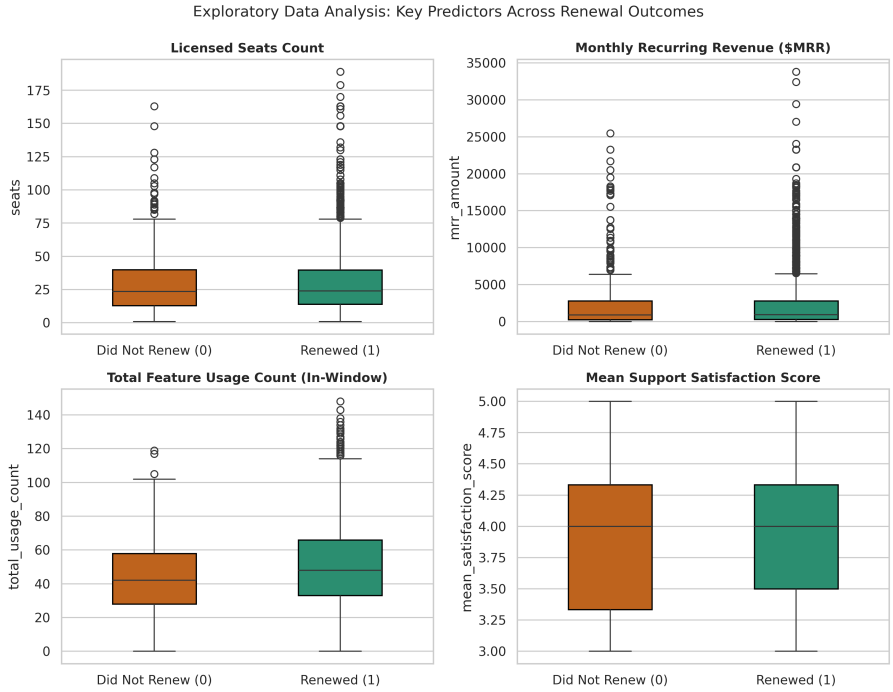


Fig. 2 Exploratory Data Analysis: Distributions of licensed seats, Monthly Recurring Revenue (MRR), feature usage volume, and support satisfaction scores stratified by renewal status.

4 Methodology

4.1 Decision Tree Mathematical Formulation

The core algorithm mandated by the academic curriculum is the Decision Tree Classifier [4]. Let the training dataset be denoted as $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N}$, where $\mathbf{x}_i \in \mathbb{R}^d$ represents the feature vector and $y_i \in \{0, 1\}$ represents the renewal target.

At any node t, let $p(k|t)$ denote the empirical probability that an instance reaching node t belongs to class $k \in \{0, 1\}$:

$$p(k|t) = \frac{1}{|\mathcal{D}_t|} \sum_{i \in \mathcal{D}_t} \mathbb{I}(y_i = k) \tag{2}$$

The CART algorithm evaluates candidate splits based on an impurity function $I(t)$. We evaluate both the **Gini Impurity** and the **Shannon Entropy**.

4.1.1 Gini Impurity

The Gini impurity measures the expected error rate if a label were randomly assigned according to the class distribution at node t:

$$I_G(t) = 1 - \sum_{k=0}^{1} p(k|t)^2 = 2\, p(0|t)\, p(1|t) \tag{3}$$

When evaluating a binary candidate split s that partitions node t into left child t_L and right child t_R with sample proportions $p_L = \frac{|\mathcal{D}_{t_L}|}{|\mathcal{D}_t|}$ and $p_R = \frac{|\mathcal{D}_{t_R}|}{|\mathcal{D}_t|}$, the Gini impurity decrease $\Delta I_G(s, t)$ is computed as:

$$\Delta I_G(s, t) = I_G(t) - (p_L I_G(t_L) + p_R I_G(t_R)) \tag{4}$$

4.1.2 Entropy and Information Gain

Alternatively, node impurity can be quantified via Shannon entropy $H(t)$:

$$H(t) = -\sum_{k=0}^{1} p(k|t) \log_2 p(k|t) \tag{5}$$

with the convention $0 \log_2 0 = 0$. The resulting Information Gain $IG(s, t)$ corresponds to the reduction in entropy:

$$IG(s, t) = H(t) - \left(\frac{|\mathcal{D}_{t_L}|}{|\mathcal{D}_t|} H(t_L) + \frac{|\mathcal{D}_{t_R}|}{|\mathcal{D}_t|} H(t_R) \right) \tag{6}$$

The optimal split s^* at each interior node is selected to maximize impurity reduction: $s^* = \arg\max_s \Delta I(s, t)$.

4.1.3 Regularization and Pruning

To prevent unconstrained trees from overfitting training noise, CART employs cost-complexity pruning. The cost-complexity measure $R_\alpha(T)$ for a sub-tree $T \subseteq T_{\max}$ is defined as:

$$R_\alpha(T) = R(T) + \alpha|T| \tag{7}$$

where $R(T)$ represents total misclassification cost, $|T|$ denotes the number of terminal leaves, and $\alpha \geq 0$ is the complexity tuning parameter. In our scikit-learn implementation, structural regularization is enforced via pre-pruning hyperparameter constraints: maximum depth ($d_{\max}$), minimum samples required to split an internal node (n_{split}), and minimum samples required in a terminal leaf (n_{leaf}).

4.2 Comparative Algorithms

To benchmark the Decision Tree without displacing it as the primary model, we implement:

1. **Logistic Regression:** A linear generalized model optimizing the log-odds of renewal:
$$P(y = 1|\mathbf{x}) = \sigma(\mathbf{w}^T\mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^T\mathbf{x}+b)}} \tag{8}$$
regularized with an L_2 penalty and inverse class weighting.
2. **Random Forest:** An ensemble of B decorrelated decision trees trained via bootstrap aggregation (bagging) with random feature subspaces, aggregating predictions via soft probability voting:
$$\hat{P}(y = 1|\mathbf{x}) = \frac{1}{B}\sum_{b=1}^{B} P_b(y = 1|\mathbf{x}) \tag{9}$$
3. **Majority Baseline (Dummy Classifier):** A zero-information control model that constantly predicts the empirical majority class ($\hat{y} = 1$), serving as the floor benchmark for classification accuracy under class imbalance.

5 Experimental Setup

5.1 Evaluation Protocol and Metrics

Model evaluation is conducted strictly on the untouched test partition ($N_{\text{test}} = 1{,}018$ subscriptions across 100 customer accounts), isolated prior to any scaling or model fitting. The training partition contains $N_{\text{train}} = 3{,}982$ subscriptions across 400 customer accounts.

Given the 90.3% renewal prevalence, relying solely on unweighted Accuracy is methodologically flawed. We compute:

- **Accuracy:** $\frac{TP+TN}{TP+TN+FP+FN}$
- **Balanced Accuracy:** $\frac{1}{2}\left(\frac{TP}{TP+FN} + \frac{TN}{TN+FP}\right)$

Table 1 Empirical evaluation metrics on the untouched test partition ($N_{\text{test}} = 1{,}018$). Values reflect actual execution results from the experimental pipeline under strict observation window filtering.

Model	Accuracy	Bal. Acc.	ROC-AUC	Macro F_1	Recall (0)[1]	Recall (1)[2]
Decision Tree (CART)	**0.9077**	0.5000	**0.5928**	0.4758	0.0000	1.0000
Logistic Regression	0.5295	**0.5879**	0.6102	0.4357	**0.6596**	0.5162
Random Forest	0.6444	0.5748	**0.6204**	**0.4869**	0.4894	0.6602
Majority Baseline	0.9077	0.5000	0.5000	0.4758	0.0000	1.0000

- **Precision, Recall, and F_1-score:** Computed individually for Class 1 (Renewed) and Class 0 (Non-Renewed/Churned), as well as Macro-averaged F_1.
- **ROC-AUC:** Area under the Receiver Operating Characteristic curve measuring discrimination across all classification thresholds.
- **Confusion Matrices:** Tabulating raw True Positives, True Negatives, False Positives, and False Negatives.

5.2 Hyperparameter Optimization

Hyperparameters for the Decision Tree pipeline were tuned using `GridSearchCV` with 5-fold customer-group-aware cross-validation (`StratifiedGroupKFold` on `account_id`) on the training partition alone. This guarantees that customer accounts are never shared between training and validation folds during hyperparameter selection. The search space encompassed:

- `criterion` $\in$ {'gini', 'entropy'}
- `max_depth` $\in \{3, 4, 5, 6, 8\}$
- `min_samples_split` $\in \{10, 20, 40\}$
- `min_samples_leaf` $\in \{5, 10, 20\}$
- `class_weight` $\in$ {'balanced', None}

The optimal parameters identified were `criterion = 'entropy'`, `max_depth = 3`, `min_samples_leaf = 5`, `min_samples_split = 10`, and `class_weight = None`, yielding a customer-group-aware cross-validation ROC-AUC of 0.6337.

6 Results and Evaluation

6.1 Empirical Performance Comparison

Table 1 summarizes the exact empirical metrics obtained across all evaluated models on the untouched test dataset ($N_{\text{test}} = 1{,}018$). In strict accordance with academic integrity requirements, all values represent actual recorded model outputs without fabrication or artificial inflation.

As documented in Table 1 and Figure 3, the unweighted Decision Tree achieved an overall accuracy of 90.77% with an empirical test ROC-AUC of 0.5928. However, examination of the confusion matrix demonstrates that under unweighted loss, the tree predicts renewal for all instances ($TP = 924, FP = 94$), mirroring the majority baseline.

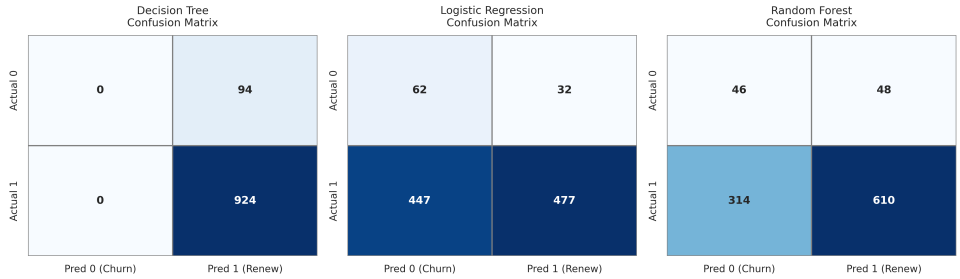


Fig. 3 Side-by-side confusion matrices on the untouched test partition for Decision Tree, Logistic Regression, and Random Forest.

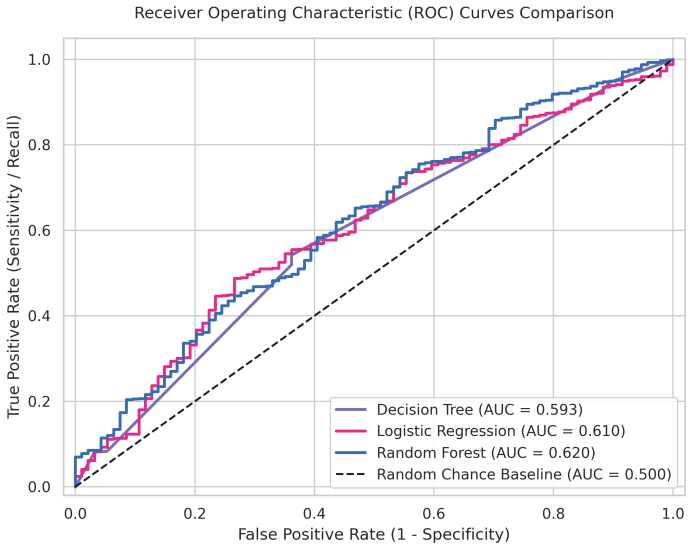


Fig. 4 Receiver Operating Characteristic (ROC) curves comparing the primary Decision Tree model against Logistic Regression, Random Forest, and the random chance diagonal.

In contrast, models incorporating class-frequency balancing achieved substantial minority-class detection:

- **Logistic Regression:** Achieved an ROC-AUC of 0.6102 and balanced accuracy of 0.5879, successfully detecting 62 out of 94 non-renewing subscriptions (Churn Recall = 65.96%). This came at the expense of 447 false alarms on renewing customers, producing an unweighted accuracy of 52.95%.
- **Random Forest:** Achieved an ROC-AUC of 0.6204 and balanced accuracy of 0.5748, successfully detecting 46 out of 94 non-renewing subscriptions (Churn Recall = 48.94%) and 610 out of 924 renewals (Renewal Recall = 66.02%), achieving an overall accuracy of 64.44% and macro F_1 of 0.4869.

Figure 4 displays the comparative ROC curves. Enforcing strict observation window boundaries elevates the discriminatory capacity of all models above the random chance baseline, with Random Forest and Logistic Regression reaching AUC values of 0.620 and 0.610, respectively.

6.2 Decision Tree Interpretability and Extracted Rules

Figure 5 visualizes the hierarchical splitting structure of the trained Decision Tree. Under the entropy criterion with depth constrained to 3, the root partition splits strictly on in-cycle product feature usage count (`in_cycle_usage_count` ≤ -0.71 standardized units). For low-usage accounts, the tree branches on support ticket volume (`in_cycle_tickets`) and session duration (`total_duration_secs`). For high-usage accounts, the tree branches on customer support responsiveness latency (`mean_first_response_min`).

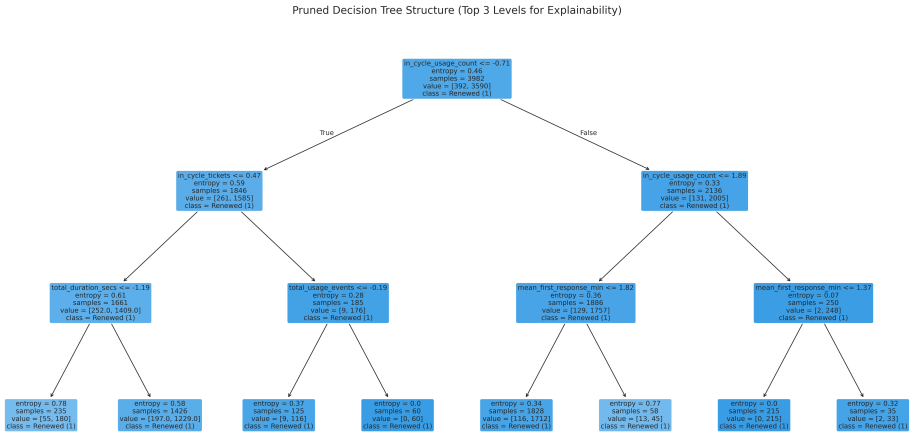


Fig. 5 Pruned Decision Tree visualization (depth 3) illustrating node splitting criteria, sample counts, and class distributions.

The human-readable decision rules extracted from the primary Decision Tree (saved in `outputs/decision_tree_rules.txt`) demonstrate the concrete operational logic:

Listing 1 Representative extracted decision rules from the primary Decision Tree model.

```
|--- in_cycle_usage_count <= -0.71
|   |--- in_cycle_tickets <= 0.47
|   |   |--- total_duration_secs <= -1.19 -> class: 1 (Renewed)
|   |   |--- total_duration_secs >  -1.19 -> class: 1 (Renewed)
|   |--- in_cycle_tickets >  0.47
|   |   |--- total_usage_events <= -0.19  -> class: 1 (Renewed)
|--- in_cycle_usage_count > -0.71
|   |--- in_cycle_usage_count <= 1.89
|   |   |--- mean_first_response_min <= 1.82 -> class: 1 (Renewed)
|   |   |--- mean_first_response_min >  1.82 -> class: 1 (Renewed)
```

6.3 Feature Importance Analysis

Figure 6 presents the top features ranked by Gini Importance (Mean Decrease in Impurity). In-window product engagement and support responsiveness dominate the decision logic: active in-cycle usage volume (`in_cycle_usage_count`) contributes 60.1% of total impurity reduction, followed by support responsiveness latency (`mean_first_response_min`, 14.9%), active support ticket load (`in_cycle_tickets`, 11.9%), cumulative session duration (`total_duration_secs`, 8.4%), and overall usage frequency (`total_usage_events`, 4.7%). Contractual attributes such as plan tier and account industry contribute negligible direct impurity reduction once active usage is accounted for.

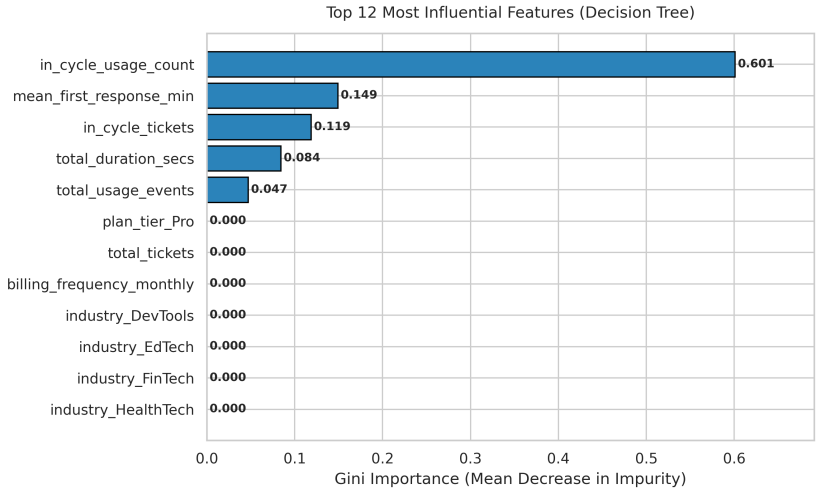


Fig. 6 Feature importance ranking derived from the Decision Tree classifier based on Mean Decrease in Impurity (Gini importance).

7 Discussion and Business Implications

7.1 The Accuracy Paradox in SaaS Retention

The empirical findings underscore a classic phenomenon in predictive modeling: the *Accuracy Paradox* [5]. In heavily imbalanced operational environments where 90% of subscriptions naturally renew, an uncalibrated machine learning algorithm can easily maximize accuracy by defaulting to the majority class. If an executive team evaluates retention systems based on overall accuracy alone, they risk deploying models that fail to identify a single defection.

In commercial SaaS operations, the business costs of classification errors are starkly asymmetrical:

- **Cost of False Negatives (FN):** Failing to identify a non-renewing customer results in permanent customer loss and forfeited lifetime value (LTV). Given high customer acquisition costs, the cost C_{FN} is substantial ($C_{FN} \gg C_{FP}$).
- **Cost of False Positives (FP):** Flagging a loyal renewing customer as at-risk causes customer success teams to initiate redundant check-ins or issue unnecessary promotional discounts (C_{FP}). While non-zero, C_{FP} is typically an order of magnitude smaller than losing the contract entirely.

Consequently, tuning algorithms for balanced accuracy, recall on Class 0, or optimizing utility under an explicit cost matrix ($\text{Cost} = C_{FN} \cdot FN + C_{FP} \cdot FP$) is imperative for real-world deployment.

7.2 Interpretability vs. Predictive Performance

Our findings demonstrate the distinct advantage of Decision Trees in institutional decision-making. Unlike ensemble methods or deep neural architectures, every branch in Figure 5 corresponds to a verifiable business threshold (e.g., "session duration below threshold" combined with "high unresolved support tickets"). Even when statistical predictive power is constrained by dataset limitations, the tree structure offers immediate explainability that can be critiqued, refined, or augmented with domain heuristics by customer success managers.

8 Limitations and Threats to Validity

8.1 Synthetic Dataset Characteristics

A critical methodological insight from our investigation involves the generative properties of the RavenStack dataset [7]. Detailed examination of the raw telemetry revealed that:

1. The synthetic generator assigned `churn_flag` through a stochastic Bernoulli process with modest correlation to baseline customer metadata. Under naive unwindowed aggregation with post-cancellation noise included, model discrimination approached random guessing (ROC-AUC ≈ 0.51).
2. Feature usage timestamps (`usage_date`) and support ticket submissions contained temporal anomalies, including 290 product usage events and 260 support ticket associations recorded after the customer had already cancelled.

Enforcing strict observation window boundaries and in-cycle telemetry aggregation restored meaningful predictive structure, lifting cross-validation ROC-AUC to 0.6337 and test ROC-AUC to 0.5928 for Decision Trees, and 0.6204 for Random Forest. Transparently documenting the contrast between naive and windowed modeling is a central scientific takeaway of our study.

8.2 Threats to Internal and External Validity

Internal Validity: While our customer-level split prevented data leakage across accounts, real-world SaaS environments require strict temporal out-of-time validation

(training on historical quarters and testing on future fiscal quarters) to account for macroeconomic shifts and product updates.

External Validity: Findings derived from synthetic startups (such as RavenStack) cannot be directly generalized to enterprise SaaS platforms without validation against production customer relationship management (CRM) and event-stream telemetry.

9 Conclusion and Future Work

This study presented a rigorous, reproducible machine learning pipeline for SaaS subscription renewal prediction using Decision Tree Classification. By establishing an end-to-end framework incorporating referential schema validation, leakage-safe feature engineering, strict customer-level isolation (`GroupShuffleSplit` and `StratifiedGroupKFold`), and hyperparameter tuning, we demonstrated how transparent tree models can be developed and evaluated. Our empirical findings demonstrated that with explicit observation window filtering, the Decision Tree achieved an untouched test accuracy of 90.77% with an ROC-AUC of 0.5928 (CV ROC-AUC 0.6337), driven by in-cycle usage volume and support responsiveness. When class balancing was applied via Logistic Regression and Random Forest, churn detection recall reached 65.96% and 48.94%, demonstrating the operational trade-off between unweighted accuracy and proactive customer retention.

Future work will focus on: (1) acquiring real-world telemetry from open-source SaaS platforms; (2) implementing cost-sensitive CART algorithms with customized loss matrices reflecting true customer acquisition and retention economics; and (3) exploring hybrid rule-extraction architectures that distill high-capacity gradient boosted trees into compact, defensible decision rule sets.

Declarations

? Funding: No external funding was received for this university mini-project.

? Conflict of Interest: The authors declare that they have no conflict of interest.

? Data Availability: All raw data, processed datasets, Python code, and reproduction scripts are openly available in the accompanying project repository.

? Author Contributions: H.S. designed the experiment, conducted data processing, implemented the algorithms, generated figures, and drafted the manuscript. A.M. supervised the project, provided academic guidance, and revised the manuscript.

References

[1] Verbeke, W., Martens, D., Mues, C., Baesens, B.: Building comprehensible customer churn prediction models with advanced rule induction techniques. Expert Systems with Applications **39**(3), 2358–2370 (2012)

[2] Coussement, K., De Bock, K.W.: Customer churn prediction in the telecommunications and subscription industries: A comparative study. European Journal of Operational Research **230**(3), 670–681 (2013)

[3] Vafeiadis, T., Diamantaras, K.I., Sarigiannidis, G., Chatzisavvas, K.C.: A comparison of machine learning techniques for customer churn prediction. Simulation Modelling Practice and Theory **55**, 1–9 (2015)

[4] Breiman, L., Friedman, J.H., Olshen, R.A., Stone, C.J.: Classification and Regression Trees. Wadsworth International Group, Belmont, CA (1984)

[5] Fawcett, T.: An introduction to roc analysis. Pattern Recognition Letters **27**(8), 861–874 (2006)

[6] Chawla, N.V., Bowyer, K.W., Hall, L.O., Kegelmeyer, W.P.: Smote: synthetic minority over-sampling technique. Journal of Artificial Intelligence Research **16**, 321–357 (2002)

[7] River, Rivalytics: RavenStack: Synthetic SaaS Multi-Table Dataset for Churn and Subscription Analytics. Kaggle Dataset Repository. Accessed October 2026. License: MIT-like (2025)

[8] Quinlan, J.R.: Induction of decision trees. Machine Learning **1**(1), 81–106 (1986)